**UNIVERSITY OF BARISHAL**

Department of Computer Science & Engineering

*Software Engineering and Information System Design Lab | CSE-3104*

# SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

# CampusFlow 3D

*A Web-Based University Transit Digital Twin Platform*

---

**Submitted by: Team StormTroopers**

Mohammod Abdullah Azrof Sadim

Araf M N Ishtiaq

Yeasin Arafat

Sajal Tikadar

Patha Sarathi Biswas

Sojib Ahmed

**Supervised by:**

Md Samsuddoha

Assistant Professor, Dept. of CSE, University of Barishal

# Team Members, Roles & Contributions

| Full Name | Student ID | Role & Responsibilities |
|---|---|---|
| Mohammod Abdullah Azrof Sadim | 230102026 | Team Leader & Systems Architect — Project coordination, Agile sprint management, system architecture, Git management, API contract definition, frontend/backend integration, presentation preparation. |
| Araf M N Ishtiaq | 230102027 | Backend Engineer — Python FastAPI backend, API routing, authentication module, simulation loop logic, REST endpoint development and Swagger/OpenAPI documentation. |
| Yeasin Arafat | 230102022 | AI & ML Engineer — Scikit-Learn model training, ETA regression, capacity classification logic, feature engineering, model serialization and FastAPI integration. |
| Sajal Tikadar | 230102031 | Frontend & Spatial UI Developer — React.js components, Mapbox GL JS 2.5D map, 3D bus model rendering, Tailwind CSS responsive design, student-facing UX. |
| Patha Sarathi Biswas | 230102013 | Database & Real-Time Systems — PostgreSQL schema, migration scripts, Socket.IO WebSocket server, real-time event broadcasting, connection management. |
| Sojib Ahmed | 230102008 | QA Tester & Deployment Engineer — Test case design, geofencing validation, WebSocket stress testing, bug tracking, Vercel and Render cloud deployment. |

# Table of Contents

# Executive Summary

CampusFlow 3D is a web-based University Transit Digital Twin Platform developed by Team StormTroopers as part of the CSE-3104 Software Engineering and Information System Design Lab course at the University of Barishal. The platform directly addresses a critical, daily challenge faced by thousands of university students: the complete absence of real-time bus tracking, predictive arrival information, and capacity awareness in the existing campus transit system.

Currently, students must physically wait at major transit hubs such as Nothullabad and Amtola More for 30 to 40 minutes with no knowledge of whether an approaching bus is full, running late, or not running at all. This 'Blind Waiting' phenomenon results in significant time waste and frustration. The 'Ghost Bus' problem — where scheduled buses fail to run without any notification — compounds the issue further.

CampusFlow 3D solves these problems by creating a real-time digital replica (a 'Digital Twin') of the university's 20-bus fleet across six routes. The MVP simulates real-time bus positions, applies AI-driven ETA predictions and capacity color-coding, and presents all information through an interactive 2.5D Mapbox map. Administrators receive a separate dashboard for fleet monitoring, SOS alert handling, and historical analytics and reporting.

The platform is built on a decoupled three-tier architecture: a React.js frontend on Vercel, a Python FastAPI backend with Socket.IO on Render, and a PostgreSQL database. Scikit-Learn powers the self-sharpening ETA model. The full MVP is built at zero cost using free-tier services, proving that high-impact transit technology is achievable without enterprise funding. This SRS document is the complete specification covering all 16 functional requirements, 22 non-functional requirements, 11 system diagrams, a test plan, API outline, database schema, and glossary.

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) provides a comprehensive, structured, and unambiguous description of the functional and non-functional requirements for CampusFlow 3D. It serves as the single authoritative reference for all phases of the software development lifecycle — design, implementation, testing, deployment, and future maintenance. It is structured in accordance with IEEE Recommended Practice for Software Requirements Specifications (IEEE 830-1998).

## 1.2 Problem Statement

The University of Barishal operates 20 buses across six routes serving students commuting from Barishal city, Jhalokathi, and Patuakhali. The system operates with zero digital infrastructure. Three clearly defined problems exist:

- Blind Waiting: Students wait 30–40 minutes at major stops with no ETA information, resulting in thousands of collective student-hours lost per semester.
- Ghost Bus Uncertainty: Scheduled buses occasionally fail to run without any notification, causing indefinite waiting.
- Capacity Blindness: Buses frequently arrive at standing-room capacity (200% of seats). Students cannot know in advance whether boarding will be possible.

CampusFlow 3D eliminates all three problems through real-time digital tracking, AI-driven predictions, and transparent capacity communication.

## 1.3 Scope

CampusFlow 3D covers real-time simulated tracking of 20 buses on 6 routes, an interactive 2.5D Mapbox map with 3D bus models, AI-driven self-sharpening ETA, Green/Yellow/Red capacity color-coding, automated geofencing for status management, Ghost Bus detection, an authenticated admin dashboard, SOS breakdown alerts with GPS coordinates, historical analytics, and exportable fleet reports.

Out of scope for this MVP: physical GPS hardware, native mobile apps, payment/ticketing, national transit integration, and offline PWA functionality.

## 1.4 Intended Audience

| Audience | Recommended Sections |
|---|---|
| Development Team | All sections — especially Sections 3, 4, 5, and Appendices B & C. |
| Academic Supervisor & Evaluators | Sections 1, 2, 3, 5, and Appendix A (Test Plan). |
| Student End Users | Section 2 (product functions) and Section 3.1 (student FRs). |
| Transport Administrators | Section 2 and Section 3.1 (admin FRs FR-06 to FR-10). |
| Future Developers & Maintainers | All sections, particularly Section 5, Appendix B (API), and Appendix C (DB Schema). |

## 1.5 Definitions, Acronyms, and Abbreviations

| Term | Definition |
|---|---|
| SRS | Software Requirements Specification — this document. |
| MVP | Minimum Viable Product — the initial deployable version of CampusFlow 3D described here. |
| Digital Twin | A real-time virtual representation of a physical system; here, a software replica of the university bus fleet. |
| ETA | Estimated Time of Arrival — predicted time (minutes) until a bus reaches its next stop. |
| Blind Waiting | Students waiting at stops with no bus arrival or capacity information. |
| Ghost Bus | A scheduled bus that does not run, causing indefinite student waiting. |
| Geofencing | Virtual geographic boundary (polygon) triggering automated status changes when a bus enters or exits. |
| Self-Sharpening ETA | ETA model that continuously improves accuracy by incorporating new travel-time data during retraining. |
| 100% Capacity (Yellow) | All designated seats occupied; standing room still available. |
| 200% Capacity (Red) | All seats occupied and all standing room exhausted — no additional passengers can board. |

| | |
|---|---|
| WebGL | Web Graphics Library — JavaScript API for hardware-accelerated 2D/3D rendering in the browser. |
| WebSocket / WSS | Full-duplex communication protocol for real-time data streaming. WSS is the encrypted variant. |
| GeoJSON | Open standard JSON format for geographic features (points, lines, polygons) used for routes and geofences. |
| JWT | JSON Web Token — signed token used for administrator session authentication. |
| FR / NFR | Functional Requirement / Non-Functional Requirement. |
| SOS Alert | Automated high-priority breakdown notification with locked GPS coordinates sent to admins. |
| Poribohon | Bangladeshi term for commercial bus transport operators — future scaling target. |
| DFD | Data Flow Diagram. |
| GLTF | GL Transmission Format — 3D model file format used for bus models on the Mapbox map. |

## 1.6 References


- CampusFlow 3D Project Proposal, Team StormTroopers, CSE-3104, University of Barishal, 2026.
- IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998.
- Mapbox GL JS Documentation — https://docs.mapbox.com/mapbox-gl-js/
- FastAPI Documentation — https://fastapi.tiangolo.com/
- Scikit-Learn Documentation — https://scikit-learn.org/stable/
- Socket.IO Documentation — https://socket.io/docs/v4/
- PostgreSQL 15 Documentation — https://www.postgresql.org/docs/15/
- React.js Documentation — https://react.dev/
- GeoJSON Specification (RFC 7946) — https://datatracker.ietf.org/doc/html/rfc7946

# 2. Overall Description

## 2.1 Product Perspective

CampusFlow 3D is a new, standalone, web-based software system introducing a completely new digital capability to the University of Barishal transit ecosystem. It is designed with a modular, decoupled architecture anticipating future integration with national commercial transit operators. The system follows a three-tier architecture: a React.js presentation tier, a Python FastAPI application tier with Socket.IO, and a PostgreSQL data tier. The AI/ML module (Scikit-Learn) is embedded within the application tier.

The geographic scope covers six university bus routes in Barishal, Jhalokathi, and Patuakhali districts. Route coordinate data is stored as GeoJSON and depot boundaries as polygon geofences in the database.

## 2.2 Product Functions Summary

| FR | Function | Description |
|---|---|---|
| FR-01 | Live Fleet Map | Real-time 2.5D Mapbox map with all active bus positions updated via WebSocket. |
| FR-02 | Bus Detail Panel | Click 3D bus model to see route, number, next stop, ETA, and capacity. |
| FR-03 | AI ETA Display | Self-sharpening, traffic-adjusted predicted arrival time per bus. |
| FR-04 | Capacity Color-Coding | Green/Yellow/Red bus color based on passenger load percentage. |
| FR-05 | Route Status List | Persistent list of all 6 routes with Active (green) or Off-Duty (red) indicators. |
| FR-06 | Admin Authentication | Secure JWT-based login for administrators. |
| FR-07 | Fleet Monitor Dashboard | Global admin map of all 20 buses with live status. |
| FR-08 | SOS Alert System | Automated breakdown alert with locked GPS sent to admin dashboard. |

| | | |
|---|---|---|
| FR-09 | Historical Analytics | Charts of crowd density per hour per route for administrators. |
| FR-10 | Fleet Impact Report | Exportable structured reports for budget and schedule decisions. |
| FR-11 | Simulation Loop | Async backend loop generating realistic simulated bus telemetry. |
| FR-12 | WebSocket Streaming | Real-time telemetry broadcast to all connected clients via Socket.IO. |
| FR-13 | Geofencing Logic | Automated In Service / Out of Service / Ghost Bus status transitions. |
| FR-14 | AI ETA & Capacity Compute | Scikit-Learn model invoked per bus per tick for ETA and capacity flag. |
| FR-15 | SOS Trigger & Log | Detect breakdown, lock GPS, log to DB, broadcast to admins. |
| FR-16 | Analytics Logging | Continuous telemetry logging and hourly aggregation. |

## 2.3 User Classes and Characteristics

### 2.3.1 Student / Passenger

The primary user class — thousands of university students accessing the system primarily from mobile browsers. They have basic digital literacy and need immediate answers to three questions: Is my bus running? When will it arrive? Can I board? The interface must be optimized for speed and visual clarity, with a maximum of 2 interactions to reach any key information.

### 2.3.2 Transport Administrator

A small group (2–5) of transport pool staff accessing the system from desktop browsers during operating hours (7 AM–9 PM). They require authenticated access, a data-dense fleet overview, and immediate SOS notification. They use the system continuously during academic operating days.

### 2.3.3 System Maintainer / Future Developer

Future contributors onboarding to maintain or scale the platform. Their needs are met by this SRS, the GitHub repository, the Swagger/OpenAPI auto-documentation at /docs, and Appendices B and C of this document.

## 2.4 Operating Environment

| Component | Specification |
|---|---|
| Student Browser | Chrome 90+, Firefox 88+, Safari 14+, Edge 90+. WebGL 1.0+ required. Minimum screen: 320px. Network: 4G LTE or campus WiFi. |
| Admin Browser | Desktop browsers (Chrome/Firefox/Edge 90+). Minimum screen: 1280px. Network: office WiFi or wired ethernet. |
| Backend Runtime | Python 3.10+ on a Linux container (Render free tier or PythonAnywhere). Single async worker process for MVP. |
| Database | PostgreSQL 14+ on a managed free-tier provider (Supabase, ElephantSQL, or Render). Storage limit: 500MB. |
| Frontend Host | Vercel global CDN for React.js static build. Automatic HTTPS. Preview deployments per branch. |
| Map Service | Mapbox GL JS browser-rendered tiles from Mapbox CDN. Free tier: 50,000 map loads/month. |

## 2.5 Design Constraints

1. Zero Budget: Entire MVP built using open-source or free-tier tools and services. No paid APIs or paid cloud resources.
2. One-Month Window: Complete MVP must be built in approximately 70 days per the academic timeline.
3. No Physical Hardware: All bus telemetry generated by software simulation (FR-11), not physical GPS devices.
4. Free-Tier Cold Starts: Render free tier has up to 50-second cold start latency after inactivity. WebSocket clients must handle reconnection gracefully (NFR-14).
5. Mapbox Quota: 50,000 map loads/month limit constrains automated testing of the frontend.
6. Cultural Capacity Calibration: The 200% Red threshold reflects local Bangladeshi transit norms where dense standing is culturally standard.
7. Browser-Only: System must function within a web browser without native app installation or special permissions beyond WebGL.

## 2.6 Assumptions and Dependencies

- GeoJSON for all 6 routes can be accurately traced from satellite imagery and field observation.
- Simulated telemetry realistically approximates real bus movement for MVP demonstration purposes.
- Mapbox maintains its free-tier API availability through the academic evaluation period.
- Vercel and Render maintain their free tiers through the evaluation period.
- Free-tier database storage (500MB) is sufficient for the MVP demonstration period with the 30-day retention policy (FR-16).
- All evaluation devices support WebGL 1.0 (safe assumption for any device from 2015 onward with an updated browser).

# 3. Specific Requirements

Section 3.1 presents all 16 Functional Requirements in full use-case format. Section 3.2 presents 22 Non-Functional Requirements organized by quality attribute.

## 3.1 Functional Requirements

### 3.1.1 Student / Passenger Module

#### FR-01: View Live Fleet Map

| Field | Description |
|---|---|
| **Requirement ID** | FR-01 |
| **Title** | View Live Fleet Map |
| **Actor(s)** | Student / Passenger |
| **Precondition** | The student's browser supports WebGL and has an active internet connection. The backend simulation loop (FR-11) is running. |
| **Main Flow** | 1. Student opens the CampusFlow 3D home URL in a browser.<br>2. Browser downloads the React.js bundle from Vercel CDN.<br>3. Application initializes the Mapbox GL JS map centered on the University of Barishal campus.<br>4. Application establishes a WebSocket connection to the backend Socket.IO server.<br>5. Application requests the 6 route GeoJSON files from the backend REST API and renders them as colored polylines on the map.<br>6. For each active bus, the backend pushes the initial position state and the frontend places a 3D bus model at the corresponding coordinates.<br>7. As new telemetry arrives via WebSocket, bus model positions update smoothly in real time. |
| **Alternative Flow** | 1. If the browser does not support WebGL, the application shows a fallback message: 'Your browser does not support 3D map rendering. Please update your browser.'<br>2. If Mapbox tiles fail within 5 seconds, a text-only fallback displays the route list and bus statuses. |
| **Postcondition** | The student sees a live, continuously updating 2.5D map of all active buses across all 6 routes. |

| | |
|---|---|
| **Exception Flow** | If the WebSocket connection drops, the application shows a reconnecting indicator and retries with exponential backoff. Bus models freeze at their last known position rather than disappearing. |
| **Priority** | High — Core student feature. |

### FR-02: View Bus Details via 3D Model Click

| Field | Description |
|---|---|
| **Requirement ID** | FR-02 |
| **Title** | View Bus Details via 3D Model Click |
| **Actor(s)** | Student / Passenger |
| **Precondition** | FR-01 has completed successfully and at least one 3D bus model is visible on the map. |
| **Main Flow** | 1. Student taps or clicks a 3D bus model on the map.<br>2. The frontend identifies the bus_id from the clicked model's metadata.<br>3. The frontend retrieves the current bus state from its WebSocket state cache.<br>4. A detail panel slides in near the selected bus showing: Route Designation, Bus Number, Next Stop name, AI-predicted ETA (FR-03), and Capacity Status (FR-04).<br>5. The detail panel updates in real time as new telemetry arrives.<br>6. Student closes the panel by tapping outside it or pressing the close icon. |
| **Alternative Flow** | 1. If two bus models overlap at low zoom, a cluster indicator is shown. The student zooms in to select individual buses. |
| **Postcondition** | The student has a live detail view for the selected bus. |
| **Exception Flow** | If the selected bus becomes 'Out of Service' while the panel is open, the panel updates to show the new status and disables the ETA field, replacing it with 'Bus currently out of service'. |
| **Priority** | High. |

### FR-03: View Self-Sharpening AI ETA

| Field | Description |
|---|---|
| **Requirement ID** | FR-03 |

| | |
|---|---|
| **Title** | View Self-Sharpening AI ETA |
| **Actor(s)** | Student / Passenger |
| **Precondition** | FR-02 detail panel is open. The selected bus has status 'In Service' and the AI/ML module has generated at least one ETA. |
| **Main Flow** | 1. The detail panel displays the AI-computed ETA (in minutes) for the selected bus's next stop from the latest telemetry packet.<br>2. The frontend subscribes to updates for this specific bus while the panel is open.<br>3. As each telemetry packet arrives, the AI/ML module recomputes ETA based on current position, speed, time-of-day, and historical travel-time distributions.<br>4. The displayed ETA updates with a smooth digit animation when the new value differs by 30 seconds or more, preventing distracting micro-fluctuations.<br>5. When the bus passes the next stop, the Next Stop field updates and ETA resets. |
| **Alternative Flow** | 1. If the student reopens the panel, ETA is fetched fresh from the latest telemetry cache. |
| **Postcondition** | The student has a continuously refined, dynamically updated arrival estimate. |
| **Exception Flow** | If the AI model cannot produce a confident ETA (insufficient historical data), the system displays 'ETA: ~X min (estimated)' with a disclaimer icon, using the static average travel time for that segment. |
| **Priority** | High. |

## FR-04: Capacity Color-Coding

| Field | Description |
|---|---|
| **Requirement ID** | FR-04 |
| **Title** | Capacity Color-Coding |
| **Actor(s)** | Student / Passenger |
| **Precondition** | FR-01 has completed. The AI/ML module is running and telemetry includes passenger load data. |
| **Main Flow** | 1. For each 'In Service' bus, the AI/ML module computes passenger load as a percentage of seated capacity. |

| | |
|---|---|
| | 2. If load < 100% of seated capacity: bus model rendered in Green, detail panel label 'Seats Available'.<br>3. If 100% ≤ load < 200% (standing room available): bus model rendered in Yellow, label 'Standing Room Only'.<br>4. If load ≥ 200% (standing room exhausted): bus model rendered in Red, label 'Full — Next Bus Recommended'.<br>5. Color updates with every telemetry packet via WebSocket, applied within one animation frame.<br>6. A text legend is always accessible via a one-tap help icon for color-blind users. |
| **Alternative Flow** | 1. If load data is temporarily unavailable, the bus renders in Gray with label 'Capacity Unknown'. |
| **Postcondition** | Every student can assess bus capacity status at a glance without opening a detail panel. |
| **Exception Flow** | If capacity classification produces a value > 300% (simulation edge case), the system caps display at Red and logs a server-side warning. |
| **Priority** | High. |

## FR-05: Persistent Route Status List

| Field | Description |
|---|---|
| **Requirement ID** | FR-05 |
| **Title** | Persistent Route Status List |
| **Actor(s)** | Student / Passenger |
| **Precondition** | The application has loaded and is receiving telemetry data. |
| **Main Flow** | 1. A persistent side panel (desktop) or collapsible bottom drawer (mobile) lists all 6 routes by name.<br>2. For each route, the system checks whether ≥1 assigned bus has status 'In Service'.<br>3. If yes: a Green filled circle is shown next to the route name.<br>4. If no: a Red filled circle is shown with label 'No Active Buses'.<br>5. Student taps a route entry: the map centers on that route and highlights its buses.<br>6. Route status updates in real time whenever a bus on that route changes status. |
| **Alternative Flow** | 1. On very narrow mobile screens (< 360px), the route list collapses by default and expands via a tap. |

| | |
|---|---|
| **Postcondition** | The student can assess active service on all routes at a glance. |
| **Exception Flow** | If backend cannot determine a route's status within 10 seconds, a Gray circle labeled 'Status Unknown' is shown until data arrives. |
| **Priority** | High. |

### 3.1.2 Administrator Module

### FR-06: Administrator Authentication

| Field | Description |
|---|---|
| **Requirement ID** | FR-06 |
| **Title** | Administrator Authentication |
| **Actor(s)** | Transport Administrator |
| **Precondition** | Administrator has valid credentials pre-configured in the ADMIN_USER table. |
| **Main Flow** | 1. Administrator navigates to /admin/login.<br>2. React frontend renders the login form.<br>3. Administrator submits username and password.<br>4. Frontend sends POST /auth/login over HTTPS.<br>5. Backend retrieves the ADMIN_USER record and verifies the password against its bcrypt hash using constant-time comparison.<br>6. On success, backend returns a signed JWT with 60-minute expiry.<br>7. Frontend stores JWT in memory (not localStorage) and redirects to /admin/dashboard.<br>8. All subsequent admin API requests include the JWT in the Authorization: Bearer header. |
| **Alternative Flow** | 1. If the session expires mid-use, the frontend detects the 401 response and redirects to /admin/login with 'Session expired — please log in again'. |
| **Postcondition** | Administrator has an authenticated session granting access to all admin-only views and API endpoints. |
| **Exception Flow** | After 5 consecutive failed login attempts, the backend enforces a 30-second lockout. A countdown timer is shown on the login form. |
| **Priority** | Critical. |

## FR-07: Monitor Entire Fleet (Admin Dashboard)

| Field | Description |
|---|---|
| **Requirement ID** | FR-07 |
| **Title** | Monitor Entire Fleet (Admin Dashboard) |
| **Actor(s)** | Transport Administrator |
| **Precondition** | FR-06 completed. WebSocket stream is operational. |
| **Main Flow** | 1. Dashboard renders a global Mapbox map covering all 6 routes.<br>2. System establishes a WebSocket subscription to all 20 buses' telemetry simultaneously.<br>3. Each bus is shown as a color-coded marker (Green/Yellow/Red per FR-04 logic).<br>4. A status sidebar lists all 20 buses by number and route with live status indicators.<br>5. Admin can filter the map view by route via a dropdown.<br>6. Admin can click any bus marker to see a telemetry panel: position, speed, load, ETA, status history.<br>7. Map and sidebar update in real time with no page refresh. |
| **Alternative Flow** | 1. If admin applies a route filter and an SOS alert fires on a different route, the system temporarily overrides the filter to show the SOS location, then restores the filter after acknowledgement. |
| **Postcondition** | Administrator has comprehensive real-time fleet visibility on a single screen. |
| **Exception Flow** | If WebSocket drops, the dashboard shows 'Connection Lost — Fleet data may be stale' and attempts automatic reconnection. |
| **Priority** | Critical. |

## FR-08: Receive and Acknowledge SOS Alert

| Field | Description |
|---|---|
| **Requirement ID** | FR-08 |
| **Title** | Receive and Acknowledge SOS Alert |
| **Actor(s)** | Transport Administrator |
| **Precondition** | FR-06 completed. FR-15 has logged the SOS and initiated the broadcast. |

| | |
|---|---|
| **Main Flow** | 1. WebSocket server broadcasts an 'sos_alert' event to all connected admin clients.<br>2. Admin dashboard renders a prominent modal showing: Bus Number, Route, Locked GPS Coordinates, mini-map preview, and Time of Alert.<br>3. An audible alert tone plays if the browser tab has audio permission.<br>4. The SOS is added to a persistent alert panel ordered by recency.<br>5. Admin dispatches assistance via offline means (radio/phone).<br>6. Admin clicks 'Acknowledge' — modal closes, status changes to 'Acknowledged' in the persistent list.<br>7. Admin later clicks 'Mark as Resolved' — backend updates SOS_ALERT.resolved_at. |
| **Alternative Flow** | 1. If modal is dismissed without clicking 'Acknowledge', it re-appears every 60 seconds until explicitly acknowledged. |
| **Postcondition** | Administrator is notified in real time and the event is permanently recorded. |
| **Exception Flow** | If multiple SOS alerts fire simultaneously, each is queued as a separate entry. The most recent unacknowledged alert appears in the modal first. |
| **Priority** | Critical. |

## FR-09: View Historical Analytics Dashboard

| Field | Description |
|---|---|
| **Requirement ID** | FR-09 |
| **Title** | View Historical Analytics Dashboard |
| **Actor(s)** | Transport Administrator |
| **Precondition** | FR-06 completed. ANALYTICS_RECORD data exists in the database (from FR-16). |
| **Main Flow** | 1. Administrator navigates to the 'Analytics' section.<br>2. Dashboard renders filter controls: Route selector, Date range picker, Hour-of-day range slider.<br>3. Admin selects filters and clicks 'Load Analytics'.<br>4. Frontend sends GET /analytics/crowd-density with parameters and JWT.<br>5. Backend queries ANALYTICS_RECORD and returns aggregated data.<br>6. Dashboard renders a bar chart (crowd density per hour) and a line chart (ETA deviation trend). |

| | |
|---|---|
| | 7. Admin can hover over chart elements to view exact values.<br>8. Admin can toggle between per-route and all-routes aggregated views. |
| **Alternative Flow** | 1. If the analytics query takes >5 seconds, a loading spinner is shown. If it times out after 15 seconds, an error with a Retry button is shown. |
| **Postcondition** | Administrator has data-driven insights into peak demand hours and ETA accuracy per route. |
| **Exception Flow** | If no data exists for the selected filters, an empty state message is shown: 'No analytics data for this period. Try a broader range.' |
| **Priority** | High. |

## FR-10: Generate and Export Fleet Impact Report

| Field | Description |
|---|---|
| **Requirement ID** | FR-10 |
| **Title** | Generate and Export Fleet Impact Report |
| **Actor(s)** | Transport Administrator |
| **Precondition** | FR-06 completed. Analytics data exists for the requested period. |
| **Main Flow** | 1. Admin navigates to the 'Reports' section.<br>2. Admin selects a report type: 'Monthly Route Utilization', 'Capacity Overload Frequency', 'ETA Accuracy Summary', or 'Custom'.<br>3. Admin specifies date range and selects one or more routes.<br>4. Admin clicks 'Generate Report'.<br>5. Backend queries ANALYTICS_RECORD and TELEMETRY_LOG, computes metrics, returns a structured JSON report.<br>6. Dashboard renders the report with summary statistics, per-route breakdown table, and charts.<br>7. Admin clicks 'Export as CSV' or 'Export as PDF' to download the file. |
| **Alternative Flow** | 1. If PDF export fails, the system falls back to CSV and notifies the admin. |
| **Postcondition** | Administrator has a structured, exportable report for university management. |
| **Exception Flow** | If data for the selected period includes gaps, the report shows a disclaimer: 'Note: Data incomplete for [date range] due to a service interruption.' |

| | |
|---|---|
| **Priority** | Medium. |

### 3.1.3 System / Automated Backend Module

### FR-11: Run Independent Bus Simulation Loop

| Field | Description |
|---|---|
| **Requirement ID** | FR-11 |
| **Title** | Run Independent Bus Simulation Loop |
| **Actor(s)** | System (Automated) |
| **Precondition** | Backend server has started. Route GeoJSON for all 6 routes is loaded. PostgreSQL is connected. |
| **Main Flow** | 1. On startup, backend initializes 20 simulated bus entities with assigned route_id, capacity values, and initial 'Out of Service' status.<br>2. Simulation loop starts as an asyncio background task, independent of the request-handling event loop.<br>3. On each tick (every 1–2 seconds): advances each 'In Service' bus's position along its GeoJSON route; applies randomized passenger load fluctuations (boarding/alighting at stops); with a low configurable probability, sets a breakdown flag for SOS simulation.<br>4. Raw position, load, and breakdown flag are emitted to the geofencing module (FR-13).<br>5. Loop runs indefinitely until the server process is stopped. |
| **Alternative Flow** | 1. If a simulated bus reaches the terminal stop, it either reverses (round-trip) or resets to origin (loop) per its ROUTE table configuration. |
| **Postcondition** | A continuous realistic stream of bus telemetry is available without physical hardware. |
| **Exception Flow** | If the asyncio task crashes, a supervisor mechanism restarts it within 5 seconds and logs the error. |
| **Priority** | Critical. |

### FR-12: Stream Telemetry via WebSocket

| Field | Description |
|---|---|
| **Requirement ID** | FR-12 |
| **Title** | Stream Telemetry via WebSocket |

| | |
|---|---|
| **Actor(s)** | System (Automated) |
| **Precondition** | FR-14 has produced a processed bus state object. At least one client has an active WebSocket connection. |
| **Main Flow** | 1. WebSocket server maintains a registry of all active connections, keyed by connection ID and route subscription.<br>2. When FR-14 produces a processed state, it is packaged as a JSON event: {bus_id, lat, lng, status, eta_minutes, capacity_flag, next_stop, timestamp}.<br>3. Server broadcasts the event to all subscribed clients via Socket.IO rooms.<br>4. Each client's frontend receives the event and triggers a re-render of the affected bus model and open detail panels.<br>5. Server sends heartbeat pings every 30 seconds; clients not responding within 10 seconds are removed from the registry. |
| **Alternative Flow** | 1. Student clients may subscribe to a subset of buses (by route) to reduce message volume on mobile connections. The server respects these via Socket.IO rooms. |
| **Postcondition** | All connected clients receive telemetry updates within the 2-second latency target (NFR-01). |
| **Exception Flow** | If a client disconnects unexpectedly, the server detects it on the next heartbeat and removes it without affecting other clients. |
| **Priority** | Critical. |

## FR-13: Apply Automated Geofencing and Status Logic

| Field | Description |
|---|---|
| **Requirement ID** | FR-13 |
| **Title** | Apply Automated Geofencing and Status Logic |
| **Actor(s)** | System (Automated) |
| **Precondition** | Raw telemetry emitted by FR-11. Depot geofence polygons are defined in the ROUTE table. |
| **Main Flow** | 1. For each bus on each tick, perform a point-in-polygon test: is the bus's (lat, lng) inside its route's depot_geofence?<br>2. If inside depot AND status is 'In Service': transition to 'Out of Service' and log.<br>3. If outside depot AND status is 'Out of Service': transition to 'In Service' and log. |

| | |
|---|---|
| | 4. If breakdown flag is set: transition to 'Breakdown' and emit a breakdown event to FR-15.<br>5. If time since last telemetry > 60 seconds (configurable): transition to 'Ghost Bus' and log.<br>6. Pass the status-updated bus data to FR-14 for AI computation. |
| **Alternative Flow** | 1. If coordinates fall exactly on the geofence boundary, retain the previous status until the next tick provides an unambiguous reading. |
| **Postcondition** | Each bus has an accurate, automatically maintained status requiring no manual admin input. |
| **Exception Flow** | If geofence polygon data for a route is missing or malformed, all buses on that route are marked 'Status Unknown' and an error is logged. Other routes continue processing uninterrupted. |
| **Priority** | Critical. |

## FR-14: Compute AI ETA and Capacity Classification

| Field | Description |
|---|---|
| **Requirement ID** | FR-14 |
| **Title** | Compute AI ETA and Capacity Classification |
| **Actor(s)** | System (Automated) |
| **Precondition** | Status-tagged bus data received from FR-13. A trained Scikit-Learn model is loaded. Bus status is 'In Service'. |
| **Main Flow** | 1. Extract feature vector: [route_id, segment_index, normalized_position_along_segment, simulated_speed, hour_of_day, day_of_week].<br>2. Feed the feature vector into the Scikit-Learn regression model (e.g., Gradient Boosting Regressor) to predict seconds to next stop.<br>3. Convert to minutes, round to one decimal, clamp to [0, 99] minutes.<br>4. Compare passenger_load to capacity_seated (Yellow threshold) and capacity_seated × 2 (Red threshold).<br>5. Assign capacity_flag: 'green', 'yellow', or 'red'.<br>6. Package processed state: {bus_id, eta_minutes, capacity_flag}.<br>7. Emit to FR-12 (WebSocket broadcast) and FR-16 (database logging) simultaneously. |
| **Alternative Flow** | 1. On server startup (before sufficient data), a pre-trained model seeded with synthetic data is used. The model is retrained nightly from accumulated TELEMETRY_LOG data. |

| | |
|---|---|
| **Postcondition** | Every 'In Service' bus has an up-to-date AI ETA and capacity classification. |
| **Exception Flow** | If the model raises an exception (malformed feature vector), the module falls back to the static average ETA for that segment, logs the exception, and continues processing the next bus without halting. |
| **Priority** | Critical. |

## FR-15: Trigger, Broadcast, and Log SOS Alert

| Field | Description |
|---|---|
| **Requirement ID** | FR-15 |
| **Title** | Trigger, Broadcast, and Log SOS Alert |
| **Actor(s)** | System (Automated) |
| **Precondition** | Breakdown flag detected by FR-13. PostgreSQL is reachable. |
| **Main Flow** | 1. FR-13 emits a breakdown event with bus_id and locked GPS coordinates.<br>2. Backend creates an SOS_ALERT record: {bus_id, latitude, longitude, status='open', triggered_at=now()}.<br>3. Backend commits the record to the database. If commit fails, the event is placed in a retry queue.<br>4. WebSocket server immediately broadcasts 'sos_alert' to all connected admin clients, bypassing route filtering.<br>5. Admin dashboard renders the SOS modal (FR-08).<br>6. When admin marks resolved, backend updates SOS_ALERT: {status='resolved', resolved_at=now()}. |
| **Alternative Flow** | 1. If no admin is connected when SOS fires, the alert is stored as 'unacknowledged'. The next admin to log in sees all unacknowledged alerts immediately. |
| **Postcondition** | Breakdown event is surfaced to admins in real time and permanently recorded. |
| **Exception Flow** | If the database write fails after 3 retry attempts, the alert is still broadcast via WebSocket. The failure is logged and a server-side error notification is triggered. |
| **Priority** | Critical. |

## FR-16: Log Historical Telemetry and Aggregate Analytics

| Field | Description |
|---|---|
| **Requirement ID** | FR-16 |
| **Title** | Log Historical Telemetry and Aggregate Analytics |
| **Actor(s)** | System (Automated) |
| **Precondition** | Processed bus state produced by FR-14. Database is reachable. |
| **Main Flow** | 1. For every processed state from FR-14, append a row to TELEMETRY_LOG: {bus_id, lat, lng, passenger_load, eta_minutes, capacity_flag, timestamp}.<br>2. A scheduled aggregation job runs every hour via APScheduler.<br>3. The job groups TELEMETRY_LOG rows by (route_id, hour_of_day, date) for the past hour.<br>4. For each group: compute avg_crowd_density = avg(passenger_load / capacity_seated) and avg_eta_deviation where actual arrival data is available.<br>5. Upsert computed values into ANALYTICS_RECORD.<br>6. Raw TELEMETRY_LOG rows older than 30 days are purged during the aggregation job to manage free-tier storage. |
| **Alternative Flow** | 1. If the aggregation job fails for a given hour (e.g., server restart), the next run detects the missing ANALYTICS_RECORD entry and re-runs aggregation for the missed period using retained TELEMETRY_LOG data. |
| **Postcondition** | Historical data is continuously available for analytics (FR-09), reports (FR-10), and nightly model retraining (FR-14). |
| **Exception Flow** | If TELEMETRY_LOG insertion fails (e.g., disk full), real-time WebSocket streaming is not interrupted — the two subsystems are independently fault-tolerant. |
| **Priority** | High. |

## 3.2 Non-Functional Requirements

### 3.2.1 Performance

| ID | Category | Requirement |
|---|---|---|
| NFR-01 | Latency | Bus position updates shall be delivered end-to-end within ≤2 seconds under normal network conditions (4G LTE or better). |

| | | |
|---|---|---|
| NFR-02 | Frame Rate | The 2.5D Mapbox map shall maintain ≥30 fps on a mid-range 2019+ mobile device during panning and zooming. |
| NFR-03 | Throughput | The backend shall sustain ≥20 simultaneous simulated telemetry streams without dropping below 1 update per 2 seconds per bus. |
| NFR-04 | Concurrent Users | The WebSocket server shall support ≥100 simultaneous client connections without exceeding the NFR-01 latency target. |
| NFR-05 | Page Load | Initial app load including first map tile render shall complete within 5 seconds on a 4G connection with warm Vercel CDN cache. |
| NFR-06 | API Response | All non-streaming REST API endpoints shall return responses within 1 second for queries covering up to 30 days of analytics. |

### 3.2.2 Usability

| ID | Category | Requirement |
|---|---|---|
| NFR-07 | Responsiveness | Student interface shall be fully usable on screens from 320px (narrow mobile) to 1920px (desktop) without horizontal scrolling. |
| NFR-08 | Accessibility | Capacity indicators shall include text labels ('Seats Available', 'Standing Room Only', 'Full') alongside color coding for color-blind users. |
| NFR-09 | Learnability | A new student shall check their bus's ETA and capacity within 60 seconds of first opening the app, without documentation. |
| NFR-10 | Navigation Depth | Any student function shall be reachable within 2 interactions (taps/clicks) from the home map screen. |
| NFR-11 | Admin Clarity | The admin dashboard shall display all 20 bus statuses on a single screen at 1280×720 without scrolling. |

### 3.2.3 Reliability and Availability

| ID | Category | Requirement |
|---|---|---|

| | | |
|---|---|---|
| NFR-12 | Uptime | The MVP system shall maintain ≥95% uptime during the academic evaluation and demonstration period. |
| NFR-13 | Ghost Bus | System shall classify a bus as 'Ghost Bus' if no telemetry is received for >60 seconds, and remove the classification within 10 seconds of signal restoration. |
| NFR-14 | Reconnection | WebSocket clients shall automatically reconnect after disconnection using exponential backoff (initial: 1s, max: 30s), without user intervention. |
| NFR-15 | SOS Reliability | SOS alerts shall be broadcast within 3 seconds of the breakdown event, even during transient database write failures (alert-first, log-second architecture). |
| NFR-16 | Data Consistency | Telemetry logging failures shall not interrupt real-time WebSocket streaming — the two subsystems shall be independently fault-tolerant. |

### 3.2.4 Security

| ID | Category | Requirement |
|---|---|---|
| NFR-17 | Authentication | The admin dashboard and all admin API endpoints shall reject unauthenticated requests with HTTP 401. |
| NFR-18 | Password Storage | Administrator passwords shall be stored as bcrypt salted hashes (work factor ≥10). Plaintext passwords shall never be persisted or logged. |
| NFR-19 | Transport Security | All production client-server communication shall use HTTPS/WSS (TLS 1.2+). HTTP shall redirect to HTTPS. |
| NFR-20 | Session Management | JWT tokens shall expire after 60 minutes. Tokens shall be held in memory only — not in localStorage — to mitigate XSS token theft. |
| NFR-21 | Input Validation | All API inputs shall be validated against Pydantic schemas. Malformed inputs return HTTP 422 without exposing internal stack traces. |

### 3.2.5 Maintainability and Portability

| ID | Category | Requirement |
|---|---|---|
| NFR-22 | Decoupling | Frontend and backend shall communicate only through documented REST and WebSocket APIs. No shared filesystem or in-process memory between tiers. |
| NFR-23 | Version Control | Codebase shall be on GitHub with feature branches and pull request reviews. The main branch shall always be deployable. |
| NFR-24 | Portability | All environment-specific config (DB URL, JWT secret, Mapbox token) shall use environment variables. No hardcoded secrets in the codebase. |
| NFR-25 | Model Retraining | The AI/ML model shall be retrainable from TELEMETRY_LOG via a single script without requiring API contract changes. |
| NFR-26 | API Docs | FastAPI's auto-generated Swagger/OpenAPI documentation shall be kept accurate and accessible at /docs in development. |

# 4. External Interface Requirements

## 4.1 User Interfaces

### 4.1.1 Student Interface

The student interface prioritizes speed and clarity. Guiding principle: a student answers 'Is my bus coming and can I board?' within 5 seconds of opening the app. Key elements: full-screen Mapbox 2.5D map as the primary canvas; floating route status drawer pinned to the bottom (mobile) or left (desktop); color-coded 3D bus models; slide-in detail panels; and a minimalist nav bar with a Help/Legend toggle.

### 4.1.2 Administrator Interface

The admin interface is designed for data density and operational efficiency. Layout: two-column dashboard with a wide main content area (map or chart) and a narrower sidebar (bus status list, SOS alerts). Semantic colors: Green = operational, Red = attention, Yellow = caution, Gray = unknown/offline. Critical alerts appear as screen-blocking modals and persistent banners.

## 4.2 Hardware Interfaces

CampusFlow 3D MVP interfaces with no physical hardware. All bus telemetry originates from FR-11 (simulation loop). The telemetry data format consumed by FR-13 and FR-14 is identical whether the source is the simulation loop or a future real hardware telemetry gateway, ensuring a clean migration path for future physical GPS integration.

## 4.3 Software Interfaces

| Interface | Technology | Protocol | Description |
|---|---|---|---|
| Frontend ↔ Backend REST | Axios → FastAPI | HTTPS | Auth, analytics queries, report generation, route/stop data. |
| Frontend ↔ Backend WS | Socket.IO Client → Server | WSS | Real-time telemetry, SOS alerts, heartbeats. |
| Backend ↔ Database | SQLAlchemy → PostgreSQL | TCP/5432 | All DB reads/writes — logging, aggregation, authentication. |

| | | | |
|---|---|---|---|
| AI/ML Module | Scikit-Learn (embedded) | In-process | ETA regression and capacity classification models loaded in FastAPI process, invoked per telemetry tick. |
| Frontend ↔ Mapbox | Mapbox GL JS CDN | HTTPS | Map tile rendering, vector layers, 3D model placement. API key via environment variable. |

## 4.4 Communication Interfaces

- REST over HTTPS: HTTP/1.1 or HTTP/2 with TLS. Used for all synchronous request-response interactions. FastAPI auto-generates OpenAPI documentation.
- WebSocket over WSS (Socket.IO): Full-duplex over TLS. Used for real-time telemetry streaming and SOS alert broadcasting. Provides automatic reconnection, room-based broadcasting, and event namespacing.
- Mapbox HTTPS: Outbound-only from the browser to Mapbox CDN. No server-side Mapbox communication — all tile requests originate from the client.

# 5. System Models

This section presents eleven system diagrams providing a complete visual specification of CampusFlow 3D's structure, data flows, data storage, component architecture, deployment topology, and dynamic behavior. Each diagram is cross-referenced to the relevant functional requirements.

## 5.1 Use Case Diagram

Figure 5.1 illustrates all three system actors and their associated use cases. Student/Passenger covers FR-01 to FR-05; Transport Administrator covers FR-06 to FR-10; System/Automated covers FR-11 to FR-16. Dashed arrows represent include/extend relationships, reflecting the dependency chain: Simulation Loop → Geofencing → AI Computation → WebSocket Streaming.

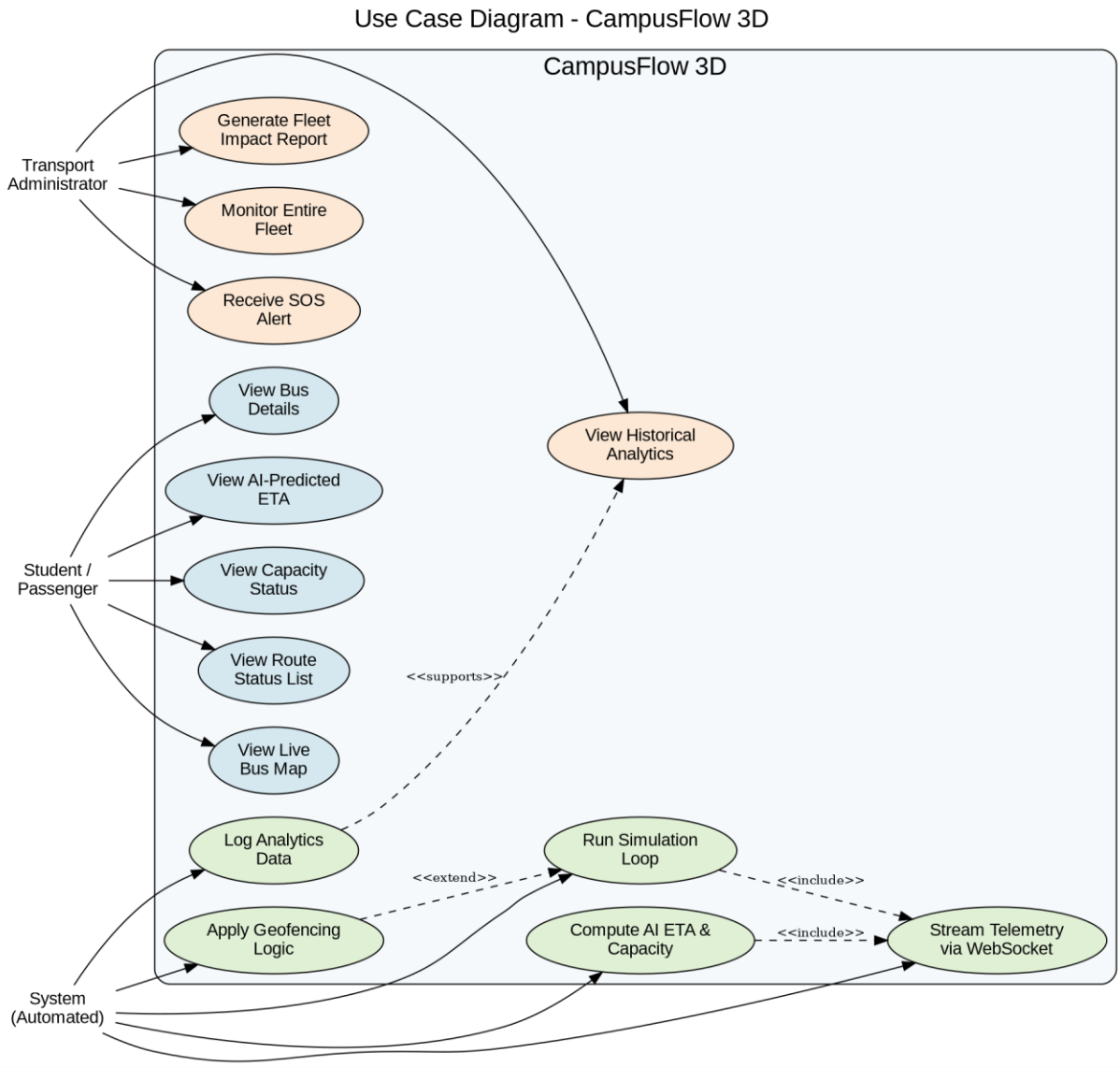


*Figure 5.1: Use Case Diagram – CampusFlow 3D*

## 5.2 System Architecture Diagram

Figure 5.2 presents the four-layer architecture: Client Layer (student and admin web apps), Application Layer (FastAPI REST, Socket.IO, Simulation Loop, Geofencing, AI/ML), Data Layer (PostgreSQL), and External Services (Mapbox CDN, Vercel, Render). This architecture implements NFR-22 (strict decoupling between frontend and backend).

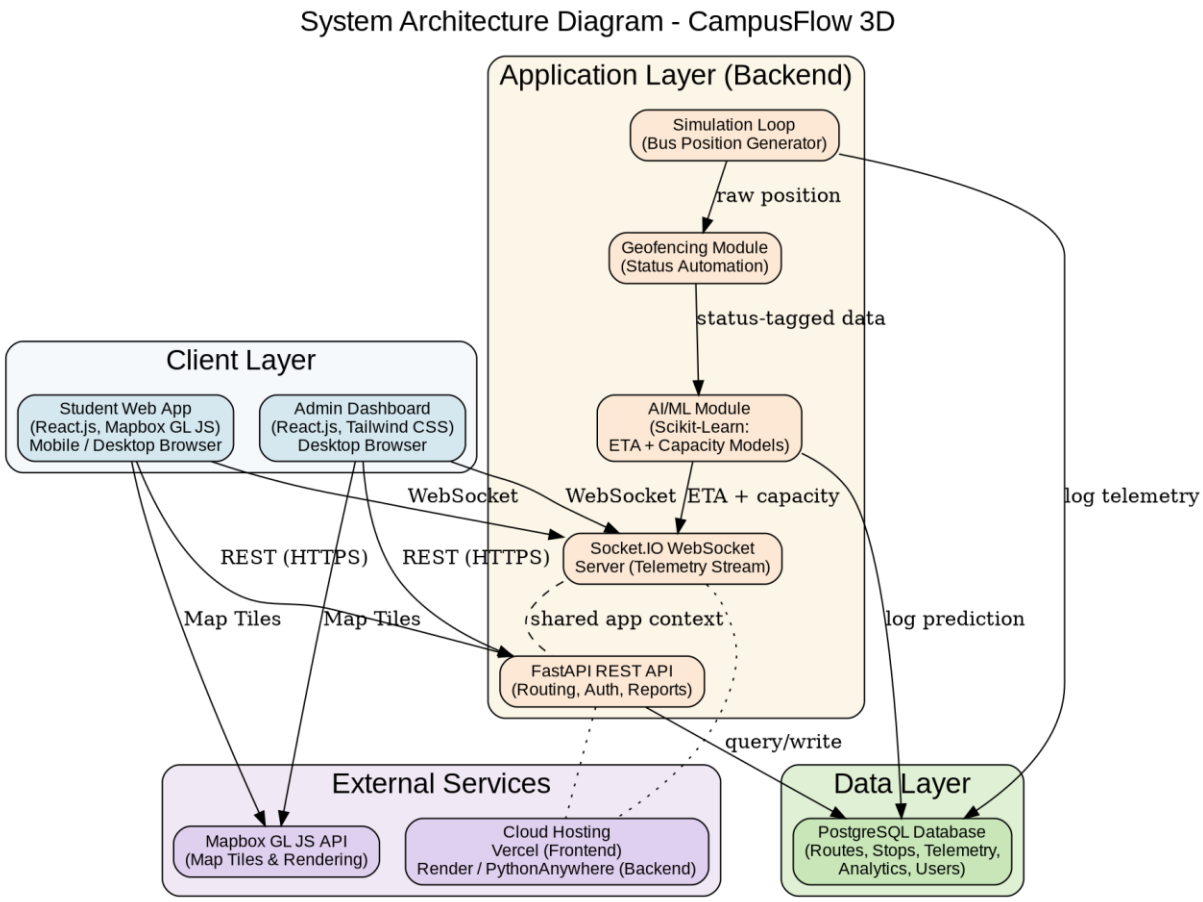


*Figure 5.2: System Architecture Diagram – CampusFlow 3D*

## 5.3 Component Diagram

Figure 5.3 decomposes the system into individual software components. On the frontend, components communicate via React Context/Zustand. On the backend, modules communicate via direct Python function calls. The WebSocket Client Hook (frontend) and WebSocket Server module (backend) form the primary real-time bridge between tiers.

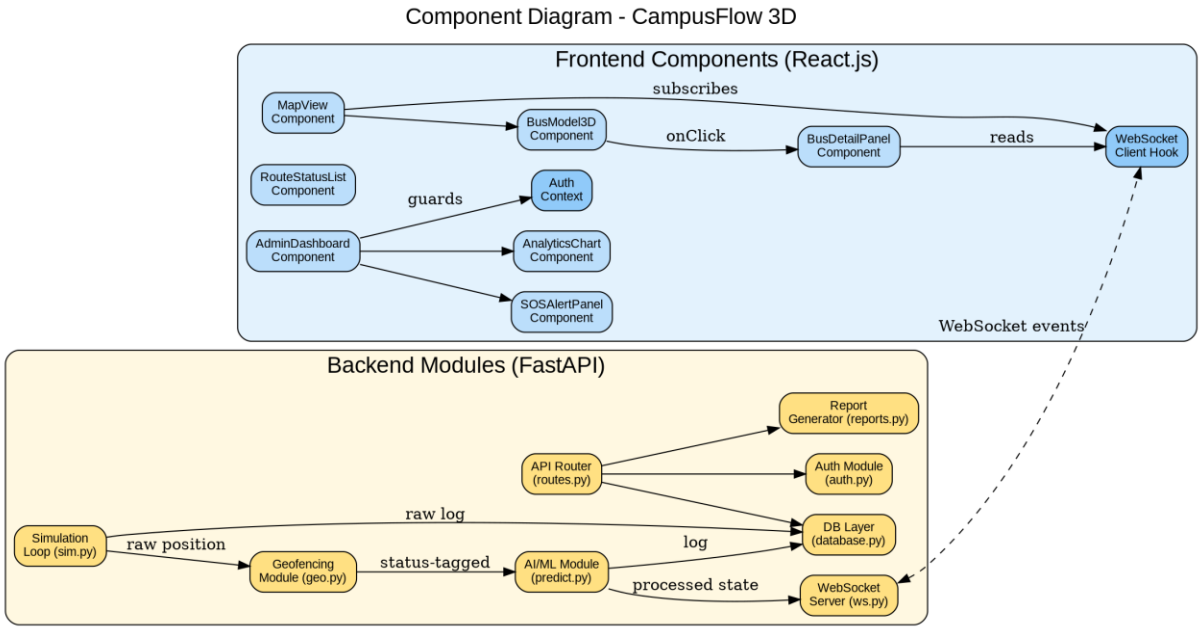


*Figure 5.3: Component Diagram – CampusFlow 3D*

## 5.4 Deployment Diagram

Figure 5.4 shows the physical and virtual infrastructure. The React.js build is served from Vercel's global CDN. FastAPI runs on a Render or PythonAnywhere free-tier Linux container. PostgreSQL is on a managed free-tier provider. Students and admins connect over the public internet via HTTPS/WSS.

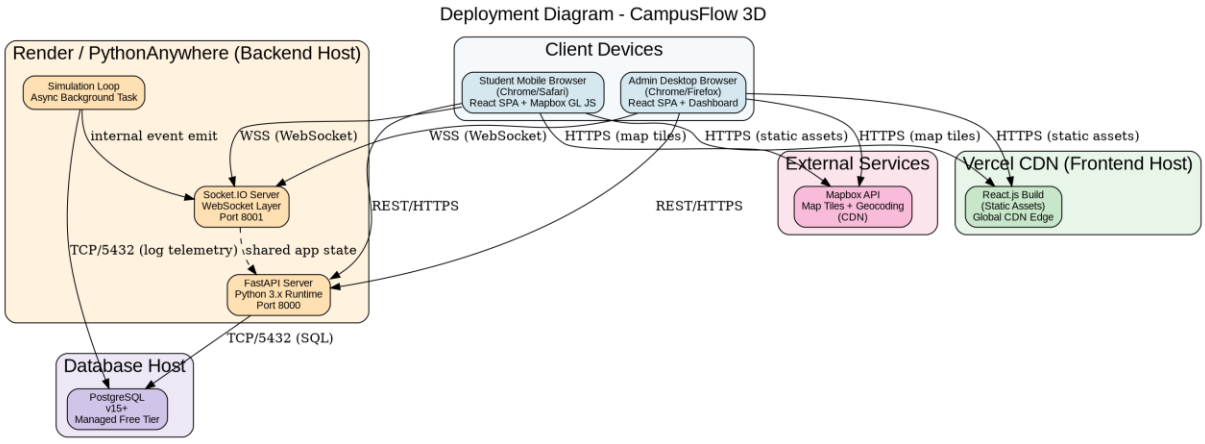


*Figure 5.4: Deployment Diagram – CampusFlow 3D*

## 5.5 Data Flow Diagrams

### 5.5.1 Level 0 – Context Diagram

Figure 5.5 treats CampusFlow 3D as a single process and shows all data exchanges with the two external actors: Student/Passenger and Transport Administrator.

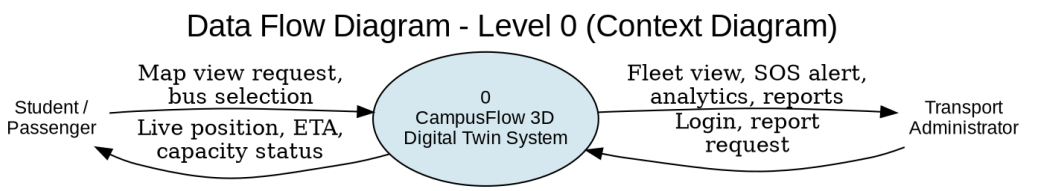


*Figure 5.5: DFD Level 0 – Context Diagram*

### 5.5.2 Level 1 – Process Decomposition

Figure 5.6 decomposes the system into six major processes and four data stores. Processes correspond to FR-11/FR-13, FR-14, FR-12, FR-15, FR-16, and FR-09/FR-10. Data stores are TELEMETRY_LOG, ROUTE & STOP, ANALYTICS_RECORD, and SOS_LOG.

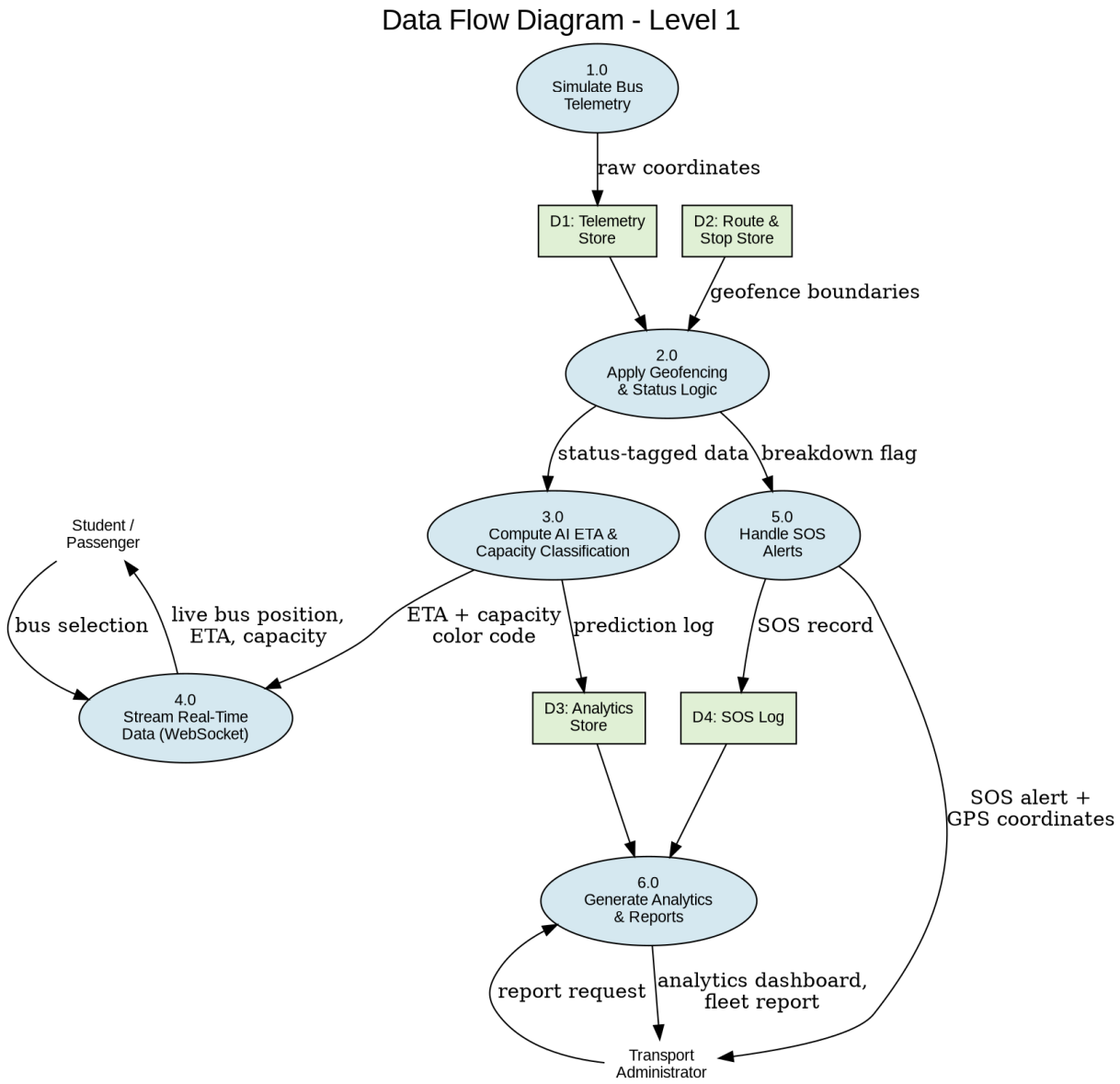


*Figure 5.6: DFD Level 1 – Process Decomposition*

## 5.6 Entity-Relationship Diagram

Figure 5.7 presents the complete database schema. The seven entities are BUS, ROUTE, STOP, TELEMETRY_LOG, ANALYTICS_RECORD, SOS_ALERT, and ADMIN_USER. ROUTE is the central hub relating to BUS, STOP, and ANALYTICS_RECORD. BUS relates to TELEMETRY_LOG and SOS_ALERT.

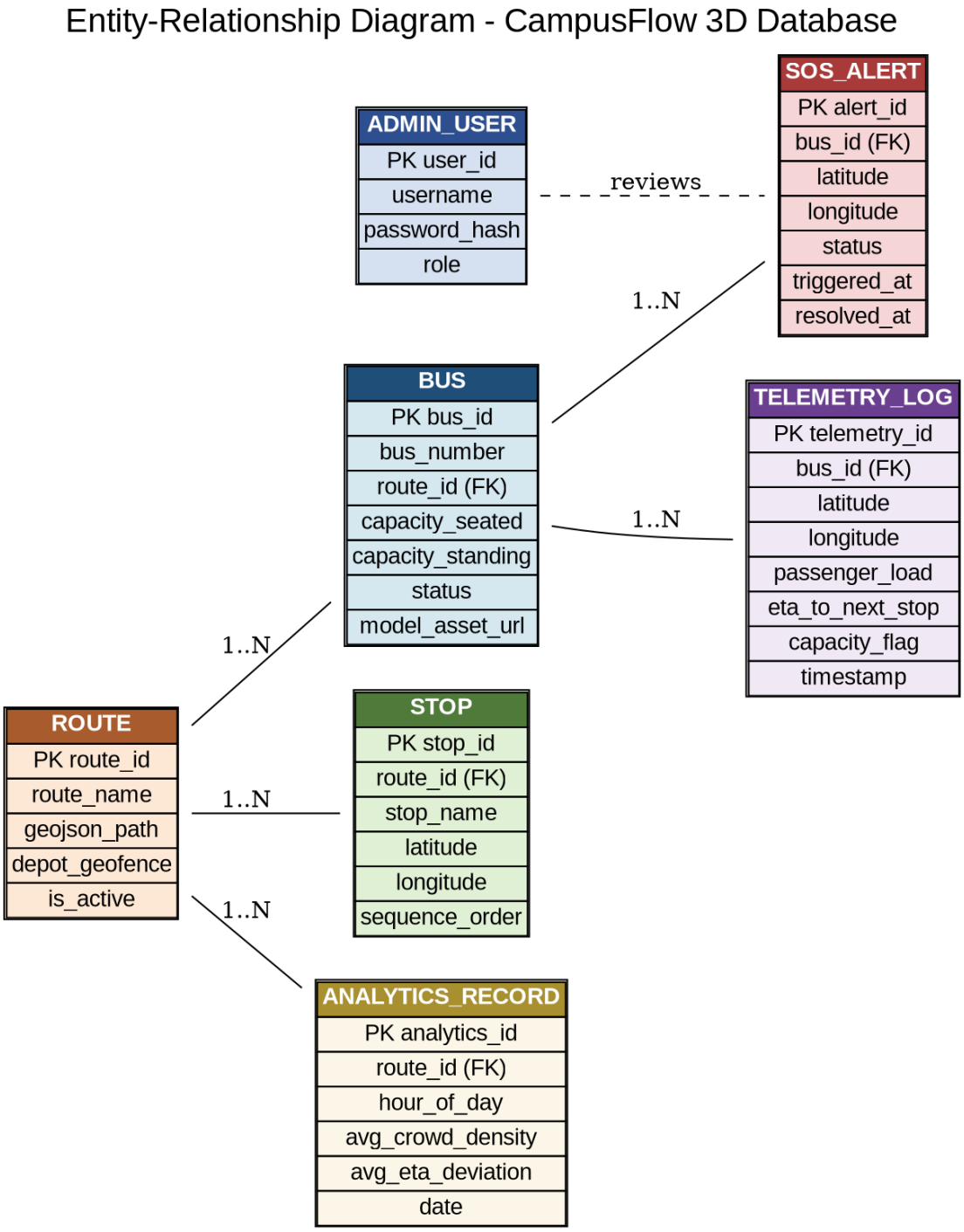


*Figure 5.7: Entity-Relationship Diagram – CampusFlow 3D Database*

### 5.6.1 Entity Descriptions

| Entity | Key Attributes | Description |
|---|---|---|

| | | |
|---|---|---|
| BUS | bus_id, bus_number, route_id (FK), capacity_seated, capacity_standing, status, model_asset_url | One of the 20 fleet buses. model_asset_url references the .gltf 3D model for frontend rendering. |
| ROUTE | route_id, route_name, geojson_path (TEXT), depot_geofence (JSONB), is_active | Stores GeoJSON route path and depot geofence polygon. JSONB used for efficient point-in-polygon queries. |
| STOP | stop_id, route_id (FK), stop_name, latitude, longitude, sequence_order | Ordered stops per route. sequence_order determines 'Next Stop' for display and ETA targeting. |
| TELEMETRY_LOG | telemetry_id (BIGSERIAL), bus_id, lat, lng, passenger_load, eta_minutes, capacity_flag, timestamp | Append-only high-volume table. Partitioned by timestamp for efficient range queries during aggregation. |
| ANALYTICS_RECORD | analytics_id, route_id (FK), hour_of_day, avg_crowd_density, avg_eta_deviation, date | Aggregated hourly statistics. One row per (route, hour, date) triple. Source for FR-09 and FR-10. |
| SOS_ALERT | alert_id, bus_id (FK), latitude, longitude, status, triggered_at, resolved_at | Immutable append-only breakdown event records. resolved_at is NULL until admin marks resolved. |
| ADMIN_USER | user_id, username (UNIQUE), password_hash, role, last_login | Administrator accounts. role distinguishes super_admin and viewer. last_login for session audit. |

## 5.7 Sequence Diagrams

### 5.7.1 Real-Time ETA and Capacity Update Flow

Figure 5.8 shows the end-to-end sequence for a single telemetry tick: Simulation Loop → Geofencing Module → AI/ML Module → WebSocket Server → Student Frontend re-renders the bus model.

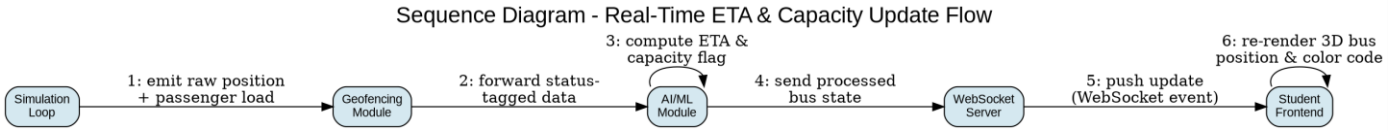


*Figure 5.8: Sequence Diagram – Real-Time ETA & Capacity Update Flow*

### 5.7.2 SOS Breakdown Alert Flow

Figure 5.9 illustrates the SOS alert lifecycle: breakdown detection in the Geofencing Module → database logging and WebSocket broadcast → administrator acknowledgement and resolution via the Admin Dashboard.

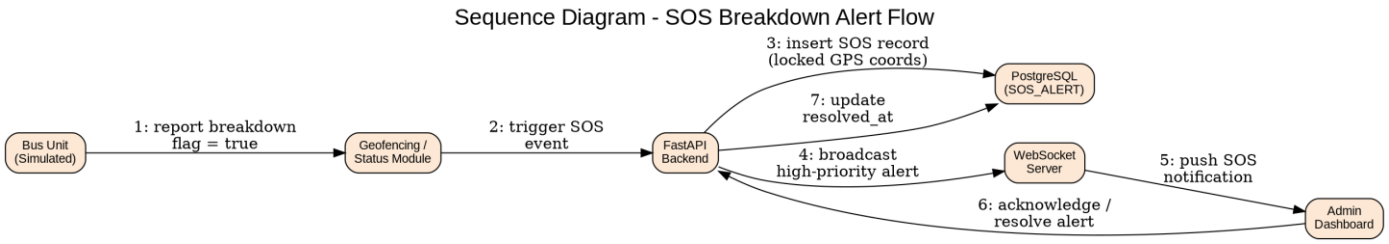


*Figure 5.9: Sequence Diagram – SOS Breakdown Alert Flow*

### 5.7.3 Administrator Authentication Flow

Figure 5.10 shows the complete admin authentication sequence: login page navigation → credential submission → bcrypt verification → JWT issuance → redirect to dashboard. Implements NFR-17 through NFR-20.

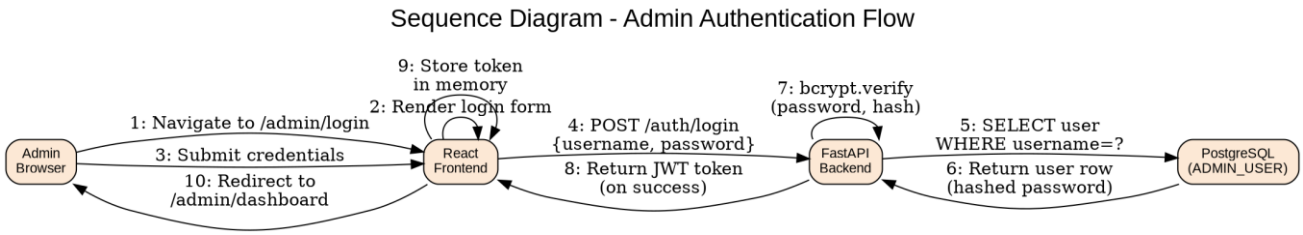


*Figure 5.10: Sequence Diagram – Admin Authentication Flow*

### 5.7.4 Fleet Report Generation Flow

Figure 5.11 shows the report generation sequence: admin selects parameters → backend queries ANALYTICS_RECORD → Report Generator computes metrics → dashboard renders output with export option.

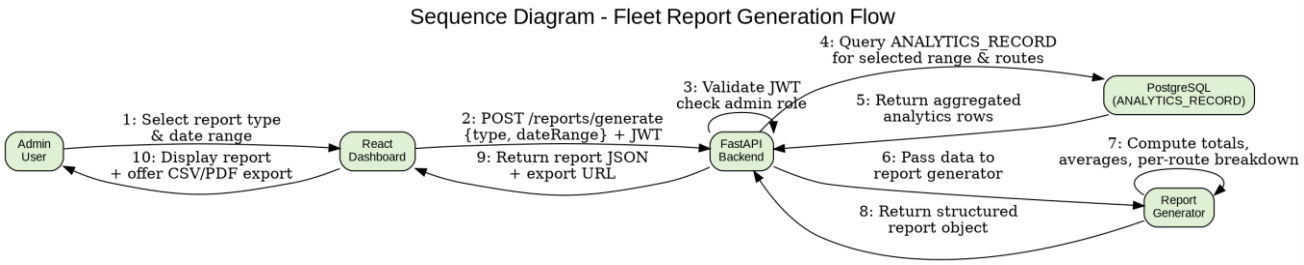


*Figure 5.11: Sequence Diagram – Fleet Report Generation Flow*

## 5.8 State Diagram – Bus Status Lifecycle

Figure 5.12 shows the complete bus status states and transitions. All transitions are automated by FR-13 (geofencing and timeout) and FR-15 (breakdown detection). No manual status changes are required for normal operation. A bus begins Out of Service at the depot, transitions to In Service when leaving the geofence, and may enter Yellow/Red capacity sub-states, Breakdown, or Ghost Bus states under exceptional conditions.

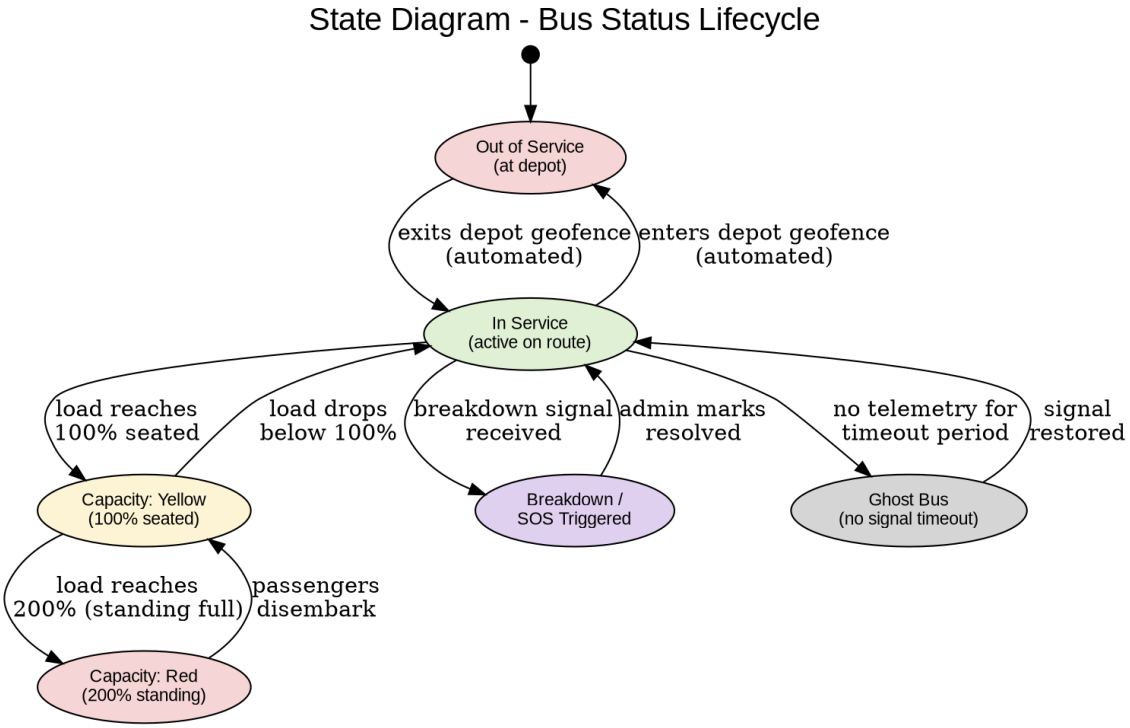


*Figure 5.12: State Diagram – Bus Status Lifecycle*

## 5.9 Activity Diagram – Student Bus-Checking Workflow

Figure 5.13 illustrates the typical end-to-end student workflow: open app → view map → select bus → check status → decide to wait or seek alternative transport. Decision points arise from FR-04 (capacity color-coding) and FR-05 (route status list).

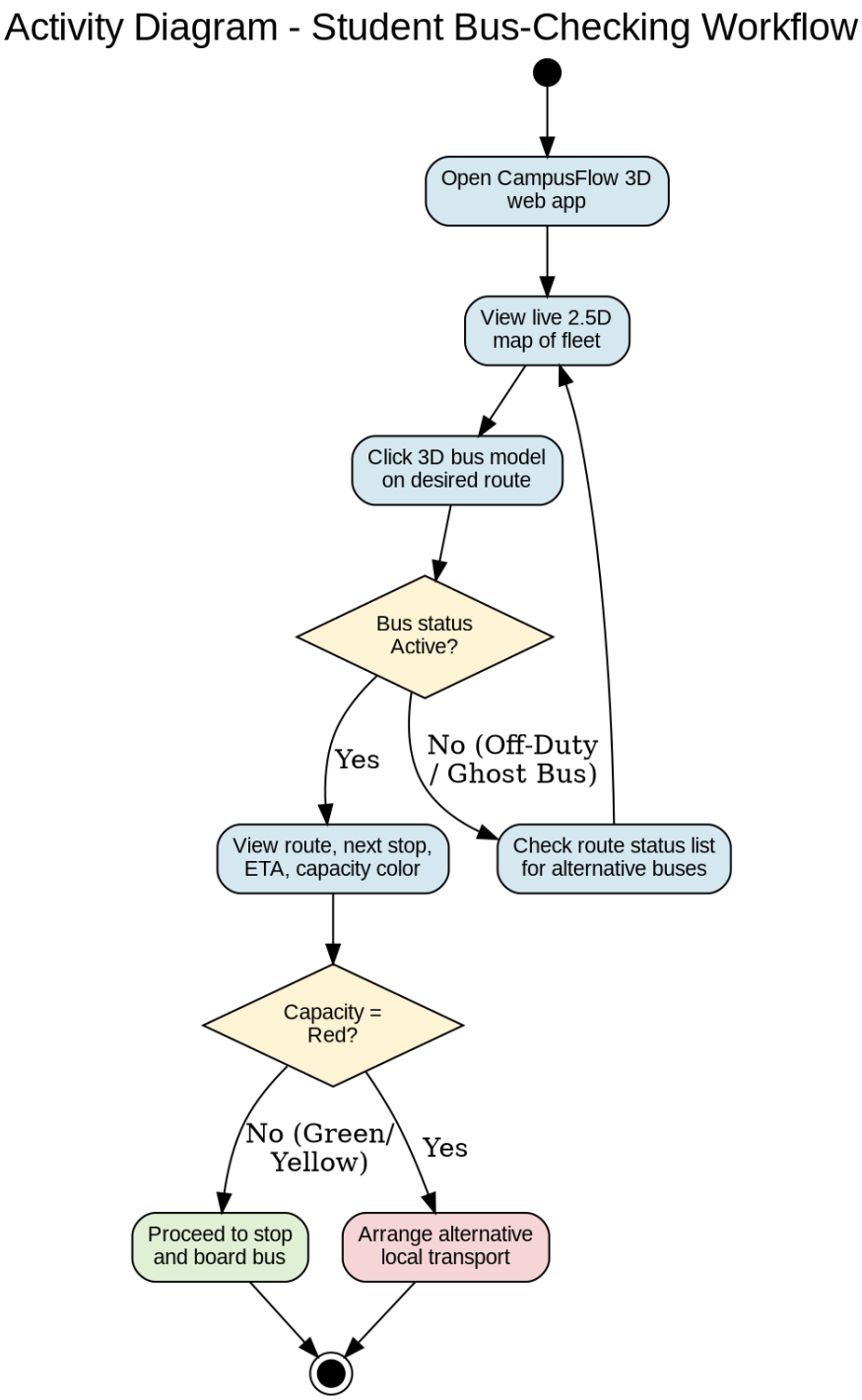


*Figure 5.13: Activity Diagram – Student Bus-Checking Workflow*

# 6. Other Requirements

## 6.1 Development Methodology

CampusFlow 3D uses the Agile Scrum framework adapted for a 6-person academic team. Work is organized into 6 sprints. Each sprint begins with planning, includes daily async check-ins, and ends with a review demo and retrospective. The product backlog is maintained as a GitHub Issues board, tagged by type, priority, and sprint. Pull requests require at least one peer review before merging into the main branch.

## 6.2 Sprint Plan

| Sprint | Duration | Goals & Deliverables |
|---|---|---|
| Sprint 1: Foundation | Days 1–10 | Requirement analysis, SRS v1.0, Git setup, React scaffold, Mapbox proof-of-concept, GeoJSON route tracing for all 6 routes. |
| Sprint 2: Backend Core | Days 11–25 | FastAPI backend, PostgreSQL schema and migrations, simulation loop (FR-11), geofencing module (FR-13), basic REST API for routes and stops. |
| Sprint 3: Real-Time UI | Days 26–40 | Socket.IO WebSocket (FR-12), 3D bus model rendering (FR-01, FR-02), real-time position updates, route status list (FR-05), mobile responsive layout. |
| Sprint 4: Intelligence | Days 41–55 | Scikit-Learn AI/ML module (FR-14), self-sharpening ETA (FR-03), Red/Yellow capacity color-coding (FR-04), telemetry logging (FR-16). |
| Sprint 5: Admin & Analytics | Days 56–65 | Admin auth (FR-06), fleet dashboard (FR-07), SOS alert system (FR-08, FR-15), analytics charts (FR-09), report generation (FR-10). |
| Sprint 6: QA & Deploy | Days 66–70 | Full integration testing, performance testing (NFR-01–06), security review (NFR-17–21), cloud deployment (Vercel + Render), final SRS v3.0, presentation prep. |

## 6.3 Tools and Technology Stack

| Category | Tool / Library | Justification |
|---|---|---|

| | | |
|---|---|---|
| Frontend Framework | React.js 18+ | Component-based UI, large ecosystem, team familiarity, excellent Mapbox GL JS integration. |
| Map Rendering | Mapbox GL JS | WebGL 2.5D rendering, free developer tier (50K loads/month), GeoJSON support, 3D model capability. |
| CSS Framework | Tailwind CSS | Utility-first, zero-runtime overhead, mobile-first responsive design, React-compatible. |
| State Management | React Context + Zustand | Lightweight, avoids Redux boilerplate. Zustand for WebSocket-driven global bus state. |
| Backend Framework | Python FastAPI | High-performance async (ASGI), automatic OpenAPI docs, Pydantic validation, asyncio background tasks. |
| WebSocket Library | Socket.IO (Python + JS) | Auto reconnection, room-based broadcasting, heartbeats, and long-polling fallback. |
| Database | PostgreSQL 14+ | ACID-compliant, JSONB for geofences, time-series partitioning for TELEMETRY_LOG, free managed tier. |
| ORM | SQLAlchemy 2.0 (async) | Async ORM support, Alembic for migrations, full PostgreSQL feature coverage. |
| AI / ML | Scikit-Learn | Mature, well-documented, excellent for tabular ETA regression and capacity classification. No GPU required. |
| Job Scheduler | APScheduler | Lightweight Python scheduler for the hourly analytics aggregation task (FR-16). |
| Auth Libraries | python-jose (JWT) + passlib (bcrypt) | Industry-standard JWT generation and bcrypt password hashing, FastAPI-idiomatic. |
| Version Control | Git + GitHub | Industry standard. GitHub Actions for CI (lint + test on pull requests). |
| Frontend Hosting | Vercel | Zero-config React deployment, global CDN, automatic HTTPS, preview URLs per branch, free tier. |

| | | |
|---|---|---|
| Backend Hosting | Render / PythonAnywhere | Free-tier Linux containers for Python, persistent process support for WebSocket, environment variable management. |

## 6.4 SWOT Analysis

| Factor | Category | Detail |
|---|---|---|
| Strength | Team Expertise | Deep understanding of decoupled architectures, WebSockets, and AI/ML integration. Team directly understands local transit challenges. |
| Strength | Cultural Fit | Capacity thresholds (200% Red) are culturally informed, making the solution immediately relevant and understandable to local users. |
| Strength | Innovation | First Digital Twin transit platform at University of Barishal. High novelty value for academic evaluation and future commercialization. |
| Weakness | No Hardware | MVP relies on simulation — no real GPS data. This limits demonstrable real-world accuracy of the ETA model. |
| Weakness | Timeline | One-month academic constraint limits depth of testing and feature polish. |
| Opportunity | Impact | Solving Blind Waiting and Ghost Bus saves measurable thousands of student-hours per semester — strong real-world impact story. |
| Opportunity | Scalability | Architecture is hardware-agnostic. Replacing the simulation loop with real GPS hardware requires minimal code changes. |
| Threat | API Limits | Mapbox free-tier quota. Exceeding 50,000 map loads/month degrades or disables the map interface. |
| Threat | Hosting Reliability | Free-tier hosting cold starts and downtime could affect demo quality. Requires a local backup demo environment. |

## 6.5 Risk Register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Mapbox free-tier limit exceeded | Medium | High | Cache tiles, restrict automated test map loads, implement static-map fallback, monitor quota dashboard. |
| Render cold start delays WebSocket | High | Medium | Configure periodic health check ping to keep service warm. Document cold start for evaluators. |
| AI ETA inaccurate (insufficient training data) | High | Medium | Pre-generate synthetic training data. Use static average ETA fallback as safety net (FR-03 exception). |
| Sprint slippage due to coursework conflicts | High | High | Prioritize core student-facing features. Defer admin analytics to last sprint if needed. Use GitHub Issues for scope control. |
| Free-tier DB storage exhaustion | Low | High | 30-day TELEMETRY_LOG retention (FR-16). Weekly storage monitoring during evaluation. |
| WebSocket degradation under peak load | Medium | Medium | Throttle mobile update frequency. Stress test with 100-client simulation before demo day. |
| Team member unavailability | Medium | Medium | Cross-train team on adjacent modules. Document all modules in README. No single points of knowledge failure. |

## 6.6 Budget Analysis

### 6.6.1 Actual MVP Cost

| Item | Cost (BDT) | Notes |
|---|---|---|
| Development labor (6 members × 70 days) | 0 | Academic project. |
| Mapbox GL JS (developer tier) | 0 | Free tier: 50,000 loads/month. |
| Vercel (frontend hosting) | 0 | Free tier: unlimited deployments. |

| | | |
|---|---|---|
| Render / PythonAnywhere (backend) | 0 | Free tier: 750 compute hours/month. |
| PostgreSQL managed database | 0 | Free tier: 500MB storage. |
| All open-source libraries | 0 | MIT / Apache 2.0 licensed. |
| TOTAL MVP COST | 0 BDT | Complete zero-budget development. |

### 6.6.2 Theoretical Enterprise Deployment (20-Bus Physical Fleet)

| Category | Description | Cost (BDT) | Type |
|---|---|---|---|
| Hardware: GPS Trackers | 20 × 4G GPS units @ BDT 8,000 | 1,60,000 | One-Time |
| Hardware: Passenger Cameras | 20 × AI counting cameras @ BDT 15,000 | 3,00,000 | One-Time |
| Hardware: Edge Compute Nodes | 20 × Raspberry Pi 4 @ BDT 9,000 | 1,80,000 | One-Time |
| SIM Data Plans | 20 × 4G SIMs @ BDT 500/month | 1,20,000 | Annual |
| Mapbox Commercial License | Commercial API license | 1,20,000 | Annual |
| AWS Cloud Hosting | EC2 + RDS + data transfer | 2,40,000 | Annual |
| Labor: Installation | Hardware installation on 20 buses | 80,000 | One-Time |
| Labor: Professional Engineering | 6-month professional dev team | 3,00,000 | One-Time |
| Contingency (10%) | Risk buffer | 1,50,000 | |
| TOTAL ENTERPRISE COST | | ~15,50,000 BDT | One-Time + Annual |

## 6.7 Future Enhancements Roadmap

| Enhancement | Version | Description |
|---|---|---|

| | | |
|---|---|---|
| Physical GPS Hardware | v2.0 | Replace simulation with real 4G GPS tracker telemetry. Hardware-agnostic architecture makes this a clean upgrade. |
| AI Passenger Counting | v2.0 | Computer-vision passenger counting cameras replacing simulated load data with real passenger counts. |
| React Native Mobile App | v2.5 | Native iOS/Android app with push notifications for 'bus approaching' alerts. |
| Bengali Language Support | v2.5 | Full Bengali (বাংলা) and English UI, user-switchable. |
| Favorited Routes & Push Alerts | v3.0 | Students save favorite routes and receive push notifications when their bus is 10 minutes away. |
| Driver Mobile App | v3.0 | Lightweight driver app for SOS reporting, route status updates, and schedule adherence. |
| National Poribohon Integration | v4.0 | Scale the platform to commercial intercity bus operators across Bangladesh. |
| Predictive Maintenance Alerts | v4.0 | Use telemetry anomalies (unusual speed drops, frequent SOS) to predict maintenance needs before breakdowns. |

# Appendix A: Test Plan

## A.1 Test Strategy

Testing is organized across six types: Unit, Integration, UI/UX, Performance, Security, and Acceptance. Unit and integration tests begin in Sprint 3 alongside new modules. Full acceptance testing occurs in Sprint 6 as a walkthrough of every FR in Section 3.1 with the supervisor.

## A.2 Test Cases

| Test ID | FR/NFR | Test Description | Expected Result |
|---|---|---|---|
| TC-01 | FR-11 | Start backend. Verify 20 bus entities initialised with correct route assignments. | 20 bus records in memory with correct route_id. |
| TC-02 | FR-11 | Run simulation for 10 ticks. Verify each bus position advances along its GeoJSON route. | Each bus (lat,lng) moves along the route polyline. |
| TC-03 | FR-13 | Place bus inside depot geofence with status 'In Service'. Run one tick. Check status. | Bus status = 'Out of Service'. |
| TC-04 | FR-13 | Place bus outside depot geofence with status 'Out of Service'. Run one tick. Check status. | Bus status = 'In Service'. |
| TC-05 | FR-13, NFR-13 | Pause telemetry for a bus for 65 seconds. Verify Ghost Bus classification. | Bus status = 'Ghost Bus' after 60s timeout. |
| TC-06 | FR-14 | Feed a known feature vector to the AI ETA model. Verify output is within [0,99] minutes. | ETA output is a float clamped to [0, 99]. |
| TC-07 | FR-14 | Set passenger_load = 110% of seated capacity. Verify capacity_flag. | capacity_flag = 'yellow'. |
| TC-08 | FR-14 | Set passenger_load = 210% of seated capacity. Verify capacity_flag. | capacity_flag = 'red'. |

| | | | |
|---|---|---|---|
| TC-09 | FR-15 | Set breakdown flag on a bus. Verify SOS_ALERT record created with locked coordinates. | New SOS_ALERT row: correct bus_id, lat, lng, status='open'. |
| TC-10 | FR-15, NFR-15 | Connect admin WebSocket client. Set breakdown flag. Measure time to 'sos_alert' event receipt. | Event received within 3 seconds. |
| TC-11 | FR-06, NFR-17 | Send GET /admin/dashboard without JWT. Check response code. | HTTP 401 Unauthorized returned. |
| TC-12 | FR-06, NFR-18 | Create admin account. Log in. Check DB — verify password is a bcrypt hash. | password_hash column contains bcrypt hash, not plaintext. |
| TC-13 | NFR-01, NFR-04 | Connect 100 WebSocket clients. Measure average telemetry event latency across all clients. | Average latency ≤ 2 seconds across all 100 clients. |
| TC-14 | NFR-05 | Load React frontend on a throttled 4G connection. Measure time to first map render. | Map renders within 5 seconds. |
| TC-15 | FR-16 | Run simulation 2 hours. Trigger aggregation job manually. Verify ANALYTICS_RECORD rows. | Rows match manually computed averages from TELEMETRY_LOG. |
| TC-16 | FR-09 | Log in as admin. Select route and 7-day range. Verify analytics chart renders. | Bar chart shows correct avg crowd density per hour. |
| TC-17 | FR-10 | Generate 'Monthly Route Utilization' report. Click 'Export as CSV'. Verify download. | CSV file downloads with correct headers and data rows. |
| TC-18 | NFR-07 | Open student interface on a 320px mobile viewport. Check for horizontal scroll. | No horizontal scrolling. All UI elements accessible. |
| TC-19 | NFR-08 | Verify capacity indicators include text labels alongside color coding. | Labels 'Seats Available', 'Standing Room Only', 'Full' are visible. |

| | | | |
|---|---|---|---|
| TC-20 | NFR-14 | Disconnect a WebSocket client forcibly. Verify automatic reconnection within 30 seconds. | Client reconnects automatically without user intervention. |

# Appendix B: REST API Endpoint Outline

The following table lists primary REST API endpoints. Full OpenAPI specification is auto-generated at /docs when the backend runs in development mode. All admin endpoints require a valid JWT Bearer token in the Authorization header.

| Method | Endpoint | Auth | FR | Description |
|---|---|---|---|---|
| POST | /auth/login | No | FR-06 | Authenticate admin. Body: {username, password}. Returns JWT on success. |
| GET | /routes | No | FR-05 | Return all routes with is_active status and assigned bus count. |
| GET | /routes/{id}/stops | No | FR-02 | Return ordered stops list for the given route. |
| GET | /buses | No | FR-01 | Return current simulated state of all buses (initial load before WebSocket connection). |
| GET | /admin/buses | Yes | FR-07 | Return full telemetry state of all 20 buses for admin fleet map. |
| GET | /admin/sos-alerts | Yes | FR-08 | Return SOS alerts filterable by status (open / acknowledged / resolved). |
| PATCH | /admin/sos-alerts/{id} | Yes | FR-08 | Update SOS alert status (acknowledge or resolve). Updates resolved_at timestamp. |
| GET | /analytics/crowd-density | Yes | FR-09 | Return aggregated crowd density per hour. Query params: route_id, date_from, date_to. |
| POST | /reports/generate | Yes | FR-10 | Generate fleet impact report. Body: {type, route_ids[], date_from, date_to}. |
| GET | /reports/{id}/export | Yes | FR-10 | Download a generated report as CSV or PDF. Query param: format=csv\|pdf. |
| GET | /health | No | NFR-12 | Health check endpoint for hosting platform uptime monitoring. |

WebSocket events via Socket.IO at /socket.io — key events: telemetry_update (server→all clients per tick), sos_alert (server→admin clients on breakdown), subscribe_route (client→server to filter by route), heartbeat ping/pong.

# Appendix C: Database Schema (PostgreSQL)

The following outlines PostgreSQL table definitions. Full migration scripts are maintained in /backend/migrations/ using Alembic.

## C.1 BUS

| Column | Type | Constraints | Description |
|---|---|---|---|
| bus_id | SERIAL | PRIMARY KEY | Auto-incremented unique bus identifier. |
| bus_number | VARCHAR(10) | NOT NULL, UNIQUE | Human-readable designation e.g. 'BU-07'. |
| route_id | INTEGER | FK → ROUTE(route_id) | The route this bus is assigned to. |
| capacity_seated | INTEGER | NOT NULL | Number of designated seats. |
| capacity_standing | INTEGER | NOT NULL | Max additional standing passengers (typically = capacity_seated). |
| status | VARCHAR(20) | NOT NULL, DEFAULT 'Out of Service' | In Service \| Out of Service \| Breakdown \| Ghost Bus. |
| model_asset_url | TEXT | NULLABLE | URL to the .gltf 3D model for frontend rendering. |

## C.2 ROUTE

| Column | Type | Constraints | Description |
|---|---|---|---|
| route_id | SERIAL | PRIMARY KEY | Auto-incremented route identifier. |
| route_name | VARCHAR(100) | NOT NULL, UNIQUE | e.g. 'UB–Nothullabad–Rupatoli'. |
| geojson_path | TEXT | NOT NULL | Full GeoJSON LineString of the route path. |

| | | | |
|---|---|---|---|
| depot_geofence | JSONB | NOT NULL | GeoJSON Polygon defining the depot boundary for geofencing logic (FR-13). |
| is_active | BOOLEAN | NOT NULL, DEFAULT TRUE | Whether the route is currently in service. |

## C.3 STOP

| Column | Type | Constraints | Description |
|---|---|---|---|
| stop_id | SERIAL | PRIMARY KEY | Auto-incremented stop identifier. |
| route_id | INTEGER | FK → ROUTE(route_id) | The route this stop belongs to. |
| stop_name | VARCHAR(100) | NOT NULL | Human-readable stop name e.g. 'Nothullabad'. |
| latitude | DOUBLE PRECISION | NOT NULL | Stop latitude coordinate. |
| longitude | DOUBLE PRECISION | NOT NULL | Stop longitude coordinate. |
| sequence_order | SMALLINT | NOT NULL | Order of this stop along the route. Used to determine 'Next Stop'. |

## C.4 TELEMETRY_LOG

| Column | Type | Constraints | Description |
|---|---|---|---|
| telemetry_id | BIGSERIAL | PRIMARY KEY | BIGSERIAL supports high-volume append-only logging. |
| bus_id | INTEGER | FK → BUS(bus_id) | The bus this record belongs to. |
| latitude | DOUBLE PRECISION | NOT NULL | Bus latitude at this record's timestamp. |
| longitude | DOUBLE PRECISION | NOT NULL | Bus longitude at this record's timestamp. |

| | | | |
|---|---|---|---|
| passenger_load | INTEGER | NOT NULL | Simulated current passenger count. |
| eta_minutes | REAL | NULLABLE | AI-predicted ETA to next stop. NULL if bus is Out of Service. |
| capacity_flag | VARCHAR(10) | NOT NULL | 'green', 'yellow', or 'red'. |
| timestamp | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() | UTC timestamp. Partition key for performance on large date ranges. |

## C.5 ANALYTICS_RECORD

| Column | Type | Constraints | Description |
|---|---|---|---|
| analytics_id | SERIAL | PRIMARY KEY | Auto-incremented. |
| route_id | INTEGER | FK → ROUTE(route_id) | The route this aggregate covers. |
| hour_of_day | SMALLINT | NOT NULL, CHECK 0–23 | Hour of day for this aggregate period. |
| avg_crowd_density | REAL | NOT NULL | avg(passenger_load / capacity_seated) for this route, hour, and date. |
| avg_eta_deviation | REAL | NULLABLE | Avg deviation between predicted and observed ETA. NULL until actual data available. |
| date | DATE | NOT NULL | Calendar date. UNIQUE on (route_id, hour_of_day, date). |

## C.6 SOS_ALERT

| Column | Type | Constraints | Description |
|---|---|---|---|
| alert_id | SERIAL | PRIMARY KEY | Auto-incremented. |
| bus_id | INTEGER | FK → BUS(bus_id) | The bus that triggered this SOS. |

| | | | |
|---|---|---|---|
| latitude | DOUBLE PRECISION | NOT NULL | Locked GPS latitude at time of breakdown. |
| longitude | DOUBLE PRECISION | NOT NULL | Locked GPS longitude at time of breakdown. |
| status | VARCHAR(15) | NOT NULL, DEFAULT 'open' | 'open', 'acknowledged', or 'resolved'. |
| triggered_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() | UTC timestamp when breakdown was detected. |
| resolved_at | TIMESTAMPTZ | NULLABLE | UTC timestamp when admin marked resolved. NULL until then. |
| acknowledged_by | INTEGER | FK → ADMIN_USER, NULLABLE | Admin who acknowledged the alert (audit trail). |

## C.7 ADMIN_USER

| Column | Type | Constraints | Description |
|---|---|---|---|
| user_id | SERIAL | PRIMARY KEY | Auto-incremented. |
| username | VARCHAR(50) | NOT NULL, UNIQUE | Login username. |
| password_hash | TEXT | NOT NULL | bcrypt hash of the password. Plaintext never stored. |
| role | VARCHAR(20) | NOT NULL, DEFAULT 'admin' | 'super_admin' or 'viewer' (for future role-based access control). |
| last_login | TIMESTAMPTZ | NULLABLE | UTC timestamp of most recent successful login. For session audit. |

# Appendix D: Glossary

| Term | Extended Definition |
|---|---|
| Agile Scrum | Iterative development framework organising work into time-boxed sprints. Each sprint produces a potentially shippable increment. Used as the development methodology for CampusFlow 3D. |
| APScheduler | Advanced Python Scheduler — Python library for scheduling recurring jobs. Used for the hourly analytics aggregation task (FR-16). |
| Asyncio | Python's built-in library for writing concurrent code using async/await syntax. FastAPI is built on asyncio, enabling non-blocking request handling and background tasks. |
| Bcrypt | A deliberately slow password hashing function resistant to brute-force attacks. Used for administrator password storage (NFR-18). |
| Cold Start | Latency experienced when a cloud service idle for some time is reactivated on the first incoming request. Render free tier can take up to 50 seconds. Mitigated by periodic health-check pings. |
| Gradient Boosting Regressor | Scikit-Learn ensemble ML algorithm building a regression model by sequentially adding decision trees. Candidate algorithm for the ETA prediction model (FR-14). |
| GLTF | GL Transmission Format — open standard 3D model file format for efficient WebGL rendering. Used for 3D bus models on the Mapbox map (FR-01, FR-02). |
| JWT | JSON Web Token — compact, cryptographically signed token for authentication claims. Used for administrator session management (FR-06, NFR-20). |
| Mapbox GL JS | JavaScript library using WebGL for interactive vector map rendering. Provides the 2.5D map canvas and 3D model layer support for the student interface. |
| Pydantic | Python data validation library used by FastAPI to define and auto-validate request/response schemas, implementing NFR-21 automatically. |
| Point-in-Polygon | Computational geometry algorithm determining whether a (lat, lng) point lies inside a defined polygon. Used for depot geofencing in FR-13. |
| Socket.IO Room | Server-side concept grouping connected clients for targeted broadcasts. Used to send route-specific telemetry only to clients subscribed to that route. |

| | |
|---|---|
| SQLAlchemy | Python SQL toolkit and async ORM. Used with Alembic migration scripts for all PostgreSQL interactions in the FastAPI backend. |
| TLS | Transport Layer Security — cryptographic protocol for secure network communication. All production CampusFlow 3D traffic uses TLS via HTTPS/WSS (NFR-19). |
| Vercel | Cloud platform for static web apps. Hosts the React.js frontend on a global CDN with automatic HTTPS, zero-config deployment from GitHub, and preview URLs per branch. |
| Zustand | Lightweight React state management library. Used to maintain a global store of live bus states from the WebSocket stream, enabling efficient re-renders of only affected components. |
| Digital Twin | A real-time virtual model of a physical system. CampusFlow 3D's Digital Twin is the software replica of the university bus fleet, updated continuously via the simulation loop. |
| Ghost Bus | A bus that fails to run on its scheduled route, causing indefinite student waiting. Detected in CampusFlow 3D by telemetry timeout logic in FR-13 (NFR-13). |

# Document Approval and Sign-Off

This Software Requirements Specification has been prepared by Team StormTroopers as a deliverable for the CSE-3104 Software Engineering and Information System Design Lab course at the University of Barishal. It represents the complete, agreed-upon specification for the CampusFlow 3D MVP.

| Role | Name | Signature & Date |
|---|---|---|
| Team Leader & Systems Architect | Mohammod Abdullah Azrof Sadim | |
| Backend Engineer | Araf M N Ishtiaq | |
| AI & ML Engineer | Yeasin Arafat | |
| Frontend & Spatial UI Developer | Sajal Tikadar | |
| Database & Real-Time Systems | Patha Sarathi Biswas | |
| QA Tester & Deployment Engineer | Sojib Ahmed | |
| Academic Supervisor | Md Samsuddoha | |